



中国科学技术大学

面向 AI 多任务场景的资源调度系统设计与实现

Design and Implementation of Resource Scheduling System
for AI Multi-task Scenarios

姓 名： 谢博谦

学 号： SA23225380

校内导师： 赵功名副教授

企业导师： 张钟高级工程师

答辩日期： 2026 年 5 月 16 日

目录

- ① 研究背景与意义
- ② 国内外研究现状
- ③ 系统整体设计
- ④ 关键技术创新
- ⑤ 系统测试与分析
- ⑥ 总结与展望

目录

- 1 研究背景与意义
- 2 国内外研究现状
- 3 系统整体设计
- 4 关键技术创新
- 5 系统测试与分析
- 6 总结与展望

AI 多任务调度的核心矛盾

背景

人工智能大模型在 NLP、CV、智能决策等领域广泛应用，多任务调度需求呈爆发式增长。

矛盾一：AI 特性适配性差

传统调度系统无法有效处理 AI 任务的异构参数与非结构化数据，数据治理繁杂。

矛盾二：决策过度依赖人工

系统无法自动判断海量 AI 任务是否具备执行条件，依赖人工逐一启动，浪费人力资源。

矛盾三：缺乏资源互斥检测

无法识别 GPU 等 AI 临界资源的互斥冲突，任务只能低效串行执行，计算资源严重浪费。

理论意义

- 填补 AI 多任务调度领域的技术空白
- 提出面向 AI 特性的调度算法与数据适配机制
- 为异构资源互斥检测提供新思路

工程价值

- 已落地腾讯 TEG 真实业务环境
- 月稳定支撑约 200 个 AI 多任务运行
- 显著提升资源利用率，降低运维人力投入

顺应 AI 大模型规模化落地的必然需求

目录

- ① 研究背景与意义
- ② 国内外研究现状
- ③ 系统整体设计
- ④ 关键技术创新
- ⑤ 系统测试与分析
- ⑥ 总结与展望

任务调度技术发展脉络

发展历程

- 1960s-80s: 批处理 → 分布式调度
- 1990s-2010s: 智能化、可视化, 仍聚焦”多任务并发效率”
- 开源方案: Airflow、Oozie — 灵活但复杂度高, 无 AI 场景适配

核心缺陷

从未针对 AI 大模型
高算力、长周期、
多模态交互特性
进行专门优化

方案	优势	AI 适配度
人工调度	灵活	极低
脚本队列	简单	低
Airflow/Oozie	功能丰富	低 (无 AI 特性优化)
本系统	AI 深度适配	高

现有检测机制的局限

任务可用性检测—逻辑引擎

- 单调推理 → 非单调推理 → 动态自适应
- 均依赖静态规则与预定义知识，缺乏综合环境感知
- 无法分析 AI 任务的目标、资源约束、环境动态性

资源互斥检测—传统互斥算法

- Dekker → Peterson → Lamport 时间戳
- 面向传统同构资源，无法应对 GPU/TPU 等 AI 异构资源
- 缺乏对资源需求动态变化、碎片化问题的处理能力

结论

现有技术体系在 AI 多任务场景下全面失效，亟需全新设计方案。

目录

- ① 研究背景与意义
- ② 国内外研究现状
- ③ 系统整体设计**
- ④ 关键技术创新
- ⑤ 系统测试与分析
- ⑥ 总结与展望

系统五大设计原则

① 异构参数适配

- 兼容多模态、多类型 AI 任务的数据格式与传输需求

② 环境感知与决策自主化

- 实时感知系统负载与运营策略，消除人工干预，引擎自决

③ 临界资源隔离与并发安全

- 细粒度互斥检测，保障 GPU 等独占性资源的安全并发

④ 架构分层解耦

- API 接入层 / 任务调度层 / 任务执行层，标准化接口

⑤ 细粒度权限管控

- OAuth2.0 + 自研权限微服务，保障系统安全

系统总体架构



核心技术栈: FastAPI | Celery | Redis | PostgreSQL | Docker/K8s | tRPC

功能需求

API 接入层

- 任务 CRUD、权限鉴定
- 数据自动纠错与类型推断

任务调度层

- 任务/资源可用性检测
- 负载均衡、数据同步
- 容错与恢复机制

任务执行层

- 异构参数映射
- 任务敏捷编辑
- 异常捕获与处理

非功能需求

性能需求

- 高并发低延迟响应
- API 接口 <200ms

稳定性需求

- 故障自愈，数据不丢失
- 7×24 小时可靠运行

兼容性需求

- Linux/Windows 双平台
- x86/ARM 多架构支持

5 张核心表，支撑系统全部业务流程

表名	核心字段	说明
用户表	id, role	系统用户
任务组表	id, owner	任务分组
任务表	id, config	核心任务
附件表	id, url	任务附件
资源占用表	id, type	互斥检测

设计要点

- 任务表通过 JSON 字段承载异构 AI 参数
- 资源占用表支撑细粒度互斥分析
- 任务组支持批量调度与依赖管理
- 使用 SQLAlchemy ORM + Alembic 版本迁移

目录

- ① 研究背景与意义
- ② 国内外研究现状
- ③ 系统整体设计
- ④ 关键技术创新**
- ⑤ 系统测试与分析
- ⑥ 总结与展望

三大创新点总览

创新一：AI 参数画像与数据自动纠错
克服传统系统对 AI 异构参数适配不足的问题



创新二：环境感知的自动化决策引擎
消除海量任务调度中的人工触发环节



创新三：资源互斥智能分析算法
实现 AI 临界资源的安全高效并发调度

创新一：AI 参数画像与数据自动纠错

痛点

传统调度系统仅支持结构化数据，无法处理 AI 任务的异构、非结构化参数，频繁导致任务失败。

解决方案

建立大模型任务参数画像 + Pydantic 自动纠偏

- ① 参数画像：梳理各 AI 模型参数特征，建立异构数据映射机制
- ② 自动纠偏逻辑：
 - 全角标点 → 半角标点
 - 冗余符号自动清理
 - float → int 无损类型转换
 - 参数缺失智能补全
- ③ JSON/Dict 深度映射：实现多层次参数的精准传递

创新一：数据自动纠错—效果对比

改造前：手动校验

- 程序员人工逐条检查参数
- 全角/半角混用难以发现
- 类型错误导致运行时崩溃
- 平均每任务耗时 >5min

改造后：自动纠偏

- 系统自动检测并纠正参数
- 全角自动转半角
- 类型错误自动推断转换
- 平均每任务 <1s 处理

效率提升 300×

手动校验

自动纠偏

创新二：基于逻辑引擎的自动化决策

痛点

海量 AI 任务需人工逐一判断是否可执行，浪费人力且效率低下。

解决方案

构建”条件-规则”二级嵌套逻辑引擎

条件层（环境感知）

- 系统负载状态
- 运营策略配置
- 外部依赖可用性

规则层（策略驱动）

- 用户自定义调度策略
- 配置化管理，无需改代码
- 多条件组合判定

核心转变：从”人工经验驱动” → ”引擎自决”

创新二：逻辑引擎的形式化定义

条件算子 (Condition)

逻辑判定的最小原子单元，映射关系：

$$C = \{Code, Params\} \rightarrow f_{\text{logic}}(Env, Params)$$

- *Code*: 唯一标识底层判断逻辑 (如时间范围、负载阈值)
- *Params*: 运行时参数，赋予同一逻辑适配不同场景的灵活性
- *Env*: 实时采集的内外部环境状态

规则算子 (Rule)

原子条件经逻辑算子嵌套组合而成：

$$Rule = \bigvee^n \left(\bigwedge^m Condition_{i,j} \right)$$

创新二：自动化决策—效果对比

维度	传统方式	本系统
启动方式	人工逐任务点击	引擎自动判断下发
判断依据	人工经验	实时环境感知 + 规则引擎
决策时延	分钟级	毫秒级
人力投入	高（需运维值守）	低（全自动运行）
夜间/周末	需排班值守	无人值守自动运行

效果

彻底消除海量任务下发过程中对人工干预的依赖，实现 7×24 小时自动化调度。

创新三：资源互斥智能分析算法

痛点

GPU 等 AI 临界资源存在互斥冲突，传统调度方案只能保守串行执行，资源利用率低。

解决方案

“细粒度”临界资源表 + 互斥智能检测

- ① 资源注册：任务需在定义阶段声明所需临界资源清单
- ② 状态探测：实时扫描资源占用表，检测 `is_delete` 标识
- ③ 并发决策：无冲突 → 并发执行；有冲突 → 延迟重试

资源占用表： `task_id + resource_id + is_delete` → 实时快照

冲突场景

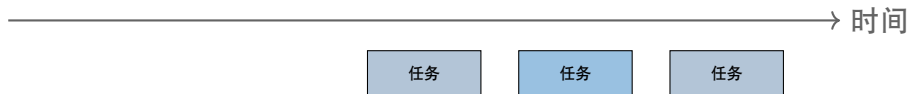
同一 GPU#1 上多个推理任务 → 串行排队

无冲突场景

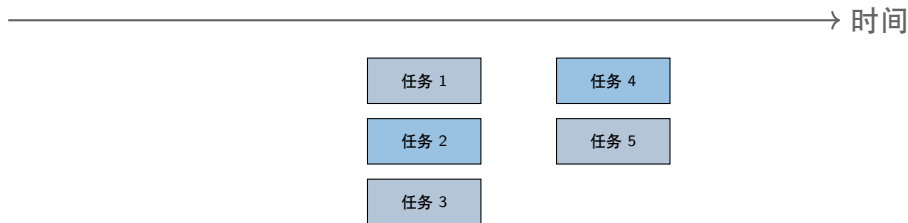
不同 GPU#1/#2 上独立任务 → 安全并发

创新三：资源互斥分析—效果对比

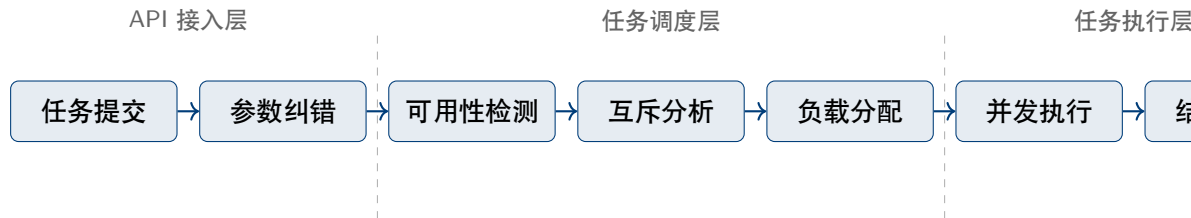
改造前：保守串行



改造后：安全并发



系统总体工作流程



三层协同：任务提交 → 参数纠错 → 可用性检测 → 互斥分析 → 负载分配 → 并发执行 → 结果回传

目录

- ① 研究背景与意义
- ② 国内外研究现状
- ③ 系统整体设计
- ④ 关键技术创新
- ⑤ 系统测试与分析
- ⑥ 总结与展望

测试环境与方法

测试环境（腾讯 TEG 生产环境）

配置项	规格
K8s 集群	3 节点 (1M + 2W)
K8s 版本	TKE 1.28.0
数据库	PostgreSQL + Redis
监控	Grafana 可视化
兼容性测试	CentOS/Debian/Win

开发环境

- CPU: i9-13900K @ 3.0GHz
- 内存: 64GB DDR5 5600MHz
- GPU: RTX 4090 (24GB)

测试工具与维度

功能测试

Postman

性能压测

Hey (Golang)

监控告警

Grafana

自动化 CI/CD

腾讯智研

功能测试结果

测试层级	测试用例数	通过率	结论
API 接入层	12 项（纠错 8 项 + 权限等 4 项）	100%	通过
任务调度层	19 项（可用性 8 项 + 资源 8 项 + 其他 3 项）	100%	通过
任务执行层	20 项（参数 8 项 + 编辑 4 项 + 异常 8 项）	100%	通过

API 接入层

全角纠偏、冗余清理、类型推断、权限拦截

任务调度层

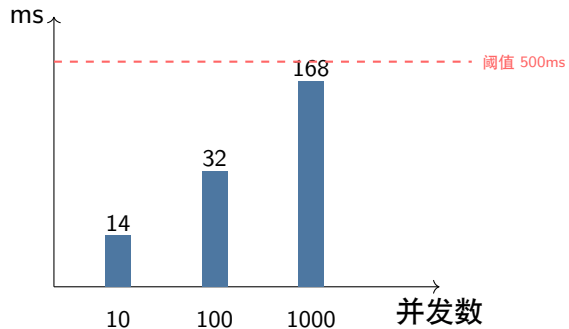
负载检测、互斥判定、等待唤醒、容错恢复

任务执行层

参数传递、撤销重启、指标监测、异常上报

性能测试结果 (Hey 压测数据)

并发数	提交	删除	编辑
10	14ms	11ms	13ms
100	32ms	28ms	30ms
1000	168ms	145ms	152ms



结论

全部压测项远低于预设阈值 (500ms), 工具: **Hey (Golang)**

稳定性与兼容性测试

稳定性测试—全部 100% 通过

场景	结果
关机重启	自动恢复, 数据无丢失
断电故障	继续未完成任务
网络中断	自动重连, 任务继续
多任务并发	无冲突, 结果正确
磁盘空间满	提示不足, 优雅降级

兼容性测试—全部 100% 通过

环境	结果
Linux x86-64	稳定无报错
Linux ARM64	稳定无报错
Windows x86-64	稳定无报错
Windows ARM64	稳定无报错

容错机制

系统采用指数退避自愈策略: 故障检测 $\rightarrow 2^{n-1}$ 秒递增重试 $\rightarrow$ 超限告警。

目录

- ① 研究背景与意义
- ② 国内外研究现状
- ③ 系统整体设计
- ④ 关键技术创新
- ⑤ 系统测试与分析
- ⑥ 总结与展望

研究工作贡献总结

① 攻克 AI 适配难题

- 参数画像 + 异构数据映射，自动纠错与类型推断
- 显著降低操作复杂度与数据处理门槛

② 突破决策自动化瓶颈

- 环境感知逻辑引擎，“条件-规则”模型
- 消除人工干预，实现 7×24 小时自动调度

③ 解决资源互斥困境

- 细粒度临界资源表 + 互斥智能检测
- 从低效串行 → 安全高效并发调度

④ 完成工程验证

- 腾讯 TEG 生产环境全面测试，月稳定支撑约 200 个 AI 任务

方向一：适配更多临界资源

- MCP 服务互斥资源对接
- 不局限于 GPU 硬件资源
- 扩展至软件服务级别资源

方向二：AI 智能数据报表

- AI 自动分析任务数据
- 多视角智能报表生成
- 节省沟通成本

紧跟 AI 大模型技术发展趋势，持续演进优化

感谢各位评委老师

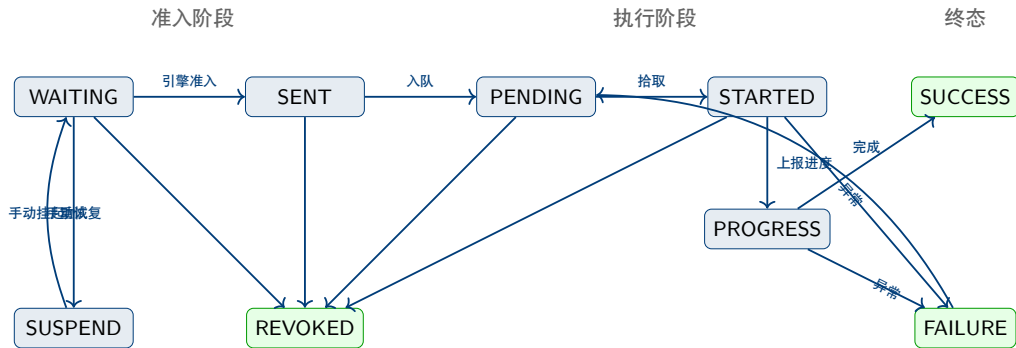
请各位老师批评指正

Q & A

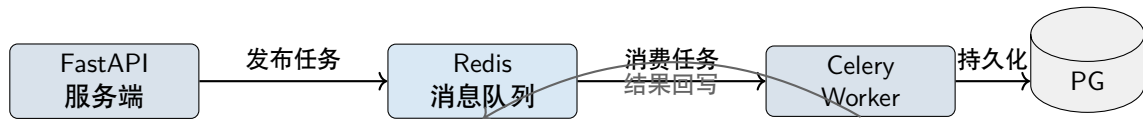
附录 A.1：技术栈总览

层次	技术选型	用途
编程语言	Python 3.10	主体开发
Web 框架	FastAPI	API 服务
前端	Vue.js	管理后台
任务队列	Celery + Redis	分布式调度
RPC 通信	tRPC（腾讯自研）	远程过程调用
数据库	PostgreSQL	数据持久化
ORM	SQLAlchemy + Alembic	数据访问
校验	Pydantic	数据校验与纠错
鉴权	OAuth2.0 + 自研微服务	权限管控
容器化	Docker + Kubernetes(TKE)	部署运维

附录 A.2: 任务全生命周期状态机

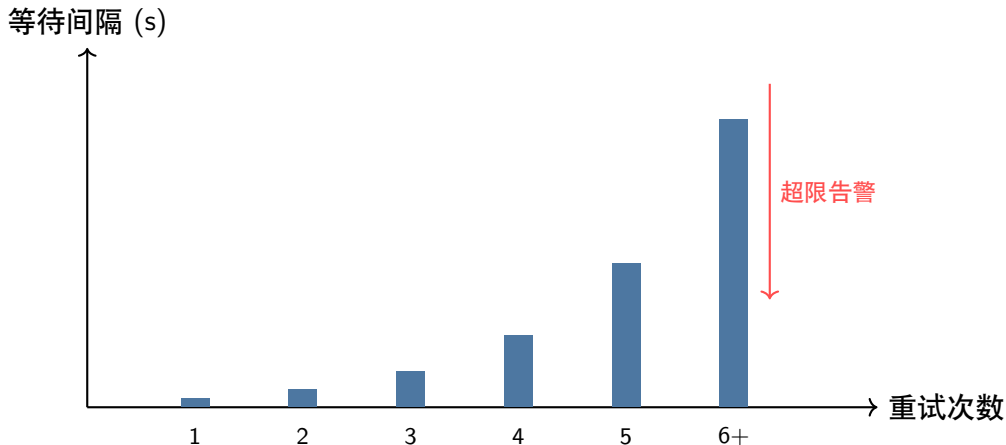


附录 A.3: Celery + Redis 数据同步机制



- API 层将任务放入 Redis 队列，立即返回响应
- Celery Workers 异步消费任务，写入 PostgreSQL
- 解耦请求处理与任务执行，保障高并发下的系统稳定性

附录 A.4: 指数退避自愈策略



- ① 任务执行失败 → 等待 2^{n-1} 秒后重试
- ② 重试次数超过阈值 → 停止重试 推送告警通知

附录 A.5: 主要参考文献



Vaswani A, et al. *Attention Is All You Need*. NeurIPS, 2017.



Radford A, et al. *Improving Language Understanding by Generative Pre-Training*. OpenAI, 2018.



Devlin J, et al. *BERT: Pre-training of Deep Bidirectional Transformers*. NAACL, 2019.



Kilburn T, et al. *The Manchester University Atlas Operating System*. The Computer Journal, 1961.



Lamport L. *Time, Clocks, and the Ordering of Events in a Distributed System*. Communications of the ACM, 1978.

完整参考文献详见学位论文